
3.1 Introduction

In addition to stacks discussed in the last chapter, a queue is another simple yet useful data structure. The queue can be found in the real world anytime one observes an assembly line in a factory where a product is being built, waiting to buy tickets for a popular new movie release, or at a drive-thru at a fast food restaurant. The first item on a conveyor belt is the first one to be completely assembled or the first person in a line is the first one to receive service. In other words, a *queue* is a first-in, first-out (*FIFO*) data structure.

A queue is also useful in computers where a queue that is perhaps most often experienced by computer users is the print queue. There are also many other queues that might not seem as obvious to computer users. For example, as one might learn in an operating systems course, that when there are two programs contending for CPU time, the operating system might choose the first one in the queue. Or as might be learned in an interfacing course, a low-level program after servicing a foreground interrupt might put information in a foreground-background queue for subsequent processing in background. There are also a number of other types of queues such as priority queues (discussed in Chap. 10), multilevel queues, and multilevel feedback queues, but this chapter will discuss a simple queue to provide a strong foundation to facilitate learning of these other types of queues in the future. Suffice it to say, queues are a very important data structure in the field of computer science.

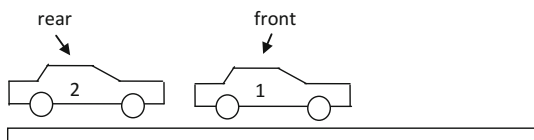
Unlike a stack, where there is only one access point at the top of the stack, there are two access points in a queue, the front and the rear of the queue. Further, whereas data can be pushed or popped from the top of the stack, items can only

enter at the rear of a queue and be removed from the front of the queue. The entering is known as an *enqueue* operation and the removing is known as a *dequeue* operation. Further, just as with a stack, a queue can be full or empty.

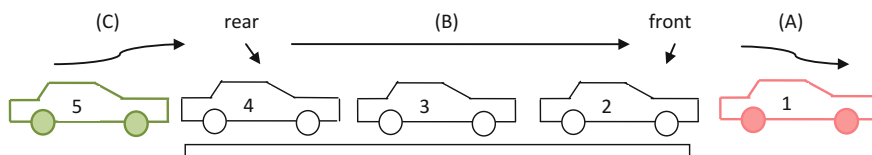
Learning some of the features of a queue can be done by looking at a simulation of an assembly line. Note that the first car in the assembly line will be both at the rear and the front of the line.



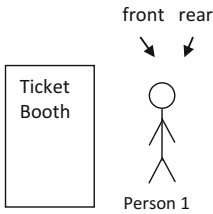
Then as the car at the front moves down the assembly line to have more parts attached to it, there is another car that can be put onto the assembly line or in other words enqueued into the queue as shown below:



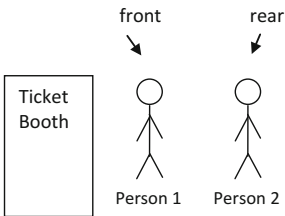
Note that in the above example, the front of the line moves as the rear remains stationary. Once the line or queue is full, (A) an assembled car will need to be dequeued, (B) the remaining cars in the queue will need to move down the line, then (C) another car can be enqueued.



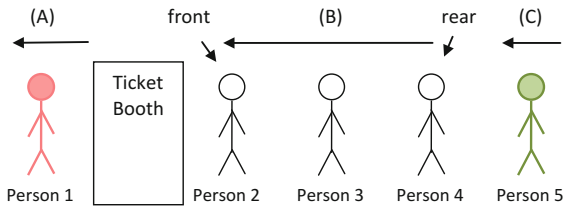
In another example, imagine waiting in line at a movie theater to get tickets for the latest blockbuster movie. As above, the first person in the line is both at the front and rear of the line.



As more people come to stand in line, the rear of the line gets longer, but the person in front stays in the same position.



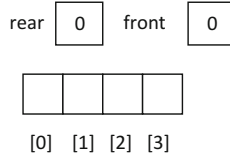
As the person in the front of the line (A) receives their ticket, proceeds to buy popcorn and enter the theatre, (B) then all of the people in the line will need to move up one position so the next person in line can purchase a ticket. Although the queue might not necessarily get full like in the assembly line example, (C) since people can continue the line out into the street, there can be a limit as to how many people will be able to purchase tickets for a seat in the theatre.



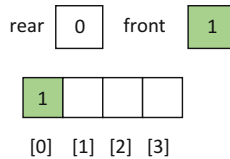
Both of the above are examples of queues, but as will be seen in the next section, having all of the cars move up each time a new one is enqueued or have all the people in line at the movie theatre move up each time a person buys a ticket and is dequeued is inefficient when it comes to analysis of the algorithm. However, there is a way to overcome this inefficiency as will be seen in the next section.

3.2 Analysis and Design

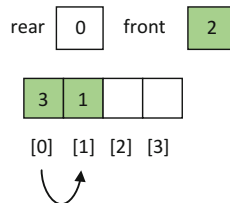
Simulating the assembly line example from the previous section and beginning to think of how to implement a queue using an array, first assume the existence of a four element array, which is small enough to make it easy to manipulate yet large enough to allow patterns to emerge. The queue will initially be empty and there also need to be indexes for the `front` and the `rear` of the queue. Both the `rear` and `front` indexes could be initialized to a 0.



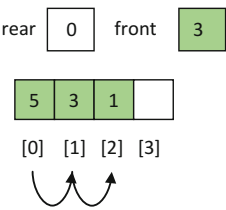
To enqueue an item such as the integer 1 at the rear of the queue, the integer 1 would be placed in position 0 and `front` would be incremented from 0 to 1 to indicate the new front of the queue and with the changes shown in green below:



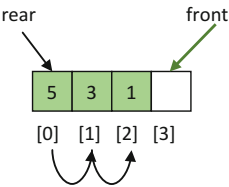
However, what would happen if one wanted to enqueue another item such as the integer 3 into the queue? In this scenario, the integer 1 would be moved to position 1, the integer 3 could then be placed in position 0, and `front` would be incremented to indicate the new front of the queue.



Then if the integer 5 were to be enqueued, the integers 3 and 1 would be moved to the right, the integer 5 placed into position 0 and `front` would again be incremented.



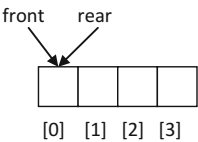
Alternatively, arrows or pointers could be used to graphically show the positions of front and rear as shown below. Either image is acceptable where the previous one is more accurate but the following one might be helpful on occasion to visualize the operations:



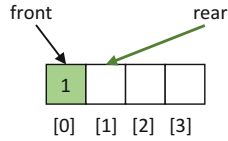
Although the above algorithm appears to work, what is a potential disadvantage to this? Imagine a one-hundred element queue with ninety-nine items in it. Before the one hundredth integer is placed into the queue, the other ninety-nine integers would need to be moved. Further, given an n element queue, then in the worst-case scenario, $n-1$ elements would need to be moved which as discussed in Chap. 1 is of order n , or $O(n)$.

However, the queue might not always be filled, so how many items would need to be moved? One could assume that on average one half the items would need to be moved, so in the case of a 100 element queue, 50 would need to be moved and in the case of an n element queue, $n/2$ items would need to be moved. Recall from Chap. 1 that $n/2$ is still a linear function of n , so it would be on average order n or $O(n)$.

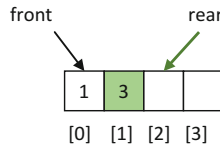
Is there a better solution to this problem and thus a more efficient algorithm? The answer is yes, where instead of moving front and the items in the queue, the rear pointer could be moved instead, similar to the movie theatre example from the previous section. Again starting with an empty queue, note that the use of arrows and that front and rear are reversed to make the drawing a little easier to read.



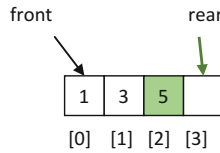
Now when the integer 1 is enqueued, `rear` has moved instead of `front`, in preparation for the next item to be enqueued all as shown in green below:



Now when 3 is enqueued, there is space for it at position 1 and `rear` is again incremented by 1.

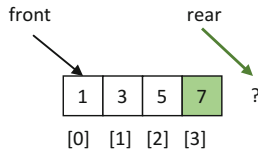


Again, when the integer 5 is enqueued, it is placed in position 2 and `rear` is moved to position 3.

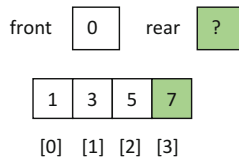


Notice so far that the advantage to this algorithm is that there is no need to move all the items in the queue when a new item is enqueued. Only `rear` needs to be changed, which takes the same constant amount of time regardless of the size of the queue. As discussed in Chap. 1, an operation like this that can be accomplished in constant time is said to be of order 1, or $O(1)$.

However, what happens if the integer 7 were to be enqueued? It would go into position 3, but what would happen to `rear`?



Switching back from using arrows to integers to help in the following discussion, the above drawing could be redrawn as follows:



Unlike the movie ticket example in the previous section, the array is of finite length and the queue has become full. Although a new larger array could be created and the contents of this queue could be copied into the new array, this would take time and the problem could occur again when that queue becomes full. An alternative is to implement what is known as a *circular queue* and have `rear` go back to position 0.

The question now is how can `rear` be made to contain a 0? The moving of `front` in the previous example and `rear` in this example up to this point should be seen as a simple increment operation, but that would not work in this case. A possible solution is to employ an `if` statement that when `rear` reaches the number 3, or $n - 1$ in the case of an n element queue, it should be set back to 0. Although this works, it would require that the value of `rear` be checked every time a new item is enqueued. The same would eventually have to be done with `front` during a dequeue operation when it reaches the end of the queue. Is there a cleaner way to solve this problem?

If one thinks back to his or her first programming class or textbook, there was an arithmetic operation that might not have seemed to be of much value at the time, but it is here that it can be quite useful. Along with the addition, subtraction, multiplication, and division operators, there was the mod operator `%` that returned the remainder when performing division. As a reminder:

```
1 % 4 = 1
2 % 4 = 2
3 % 4 = 3
```

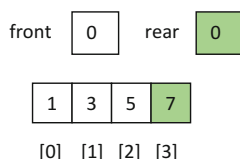
At first this might not seem too interesting because in all of the above examples the integers divided by 4 returned the original integer. However, in our problem when `rear` is incremented to 4, then the following would cause the value of `rear` to return to 0.

```
4 % 4 = 0
```

In other words, after incrementing either `front` or `rear`, instead of having to use an `if` statement to return the value back to zero, the mod operator could be used. In the cases where the value is 1 through 3, there would be no change, but

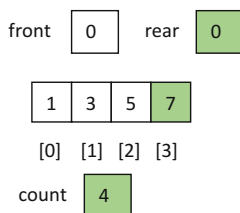
when the value becomes 4 it would revert back to 0 thus achieving the desired effect as will be seen in the code in the next section. This process could be utilized in any queue of size n .

Now after the value of 7 has been entered into the queue, the value of `rear` would be 0 as shown below:



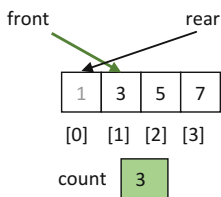
However, this does create a new problem. Note that the values of `front` and `rear` are the same as they were in the first drawing in this section. Although the reader can see that the queue is full, how will program be able to tell if the queue is full or empty? One might be tempted to just check to see if any of the cells contain values, but depending on what was previously stored in the queue this might not be a good indication whether the queue is empty. Another possible solution is to put a special value in a cell to indicate an empty cell, such as 0, but what if 0 is a valid integer that can be stored in the queue?

A solution to this problem is to use a variable `count` to indicate how many items are in the queue. If there are no items in the queue, the value of `count` would be zero. Each time an item is enqueued the value of `count` would be incremented and each time an item is dequeued the value of `count` would be decremented. In other words, when the value of `count` is zero the queue is empty and when the value of `count` equals n , the size of the array, it is full, where in the current case that value is 4. So the upgraded diagram indicating a full queue would look as follows:



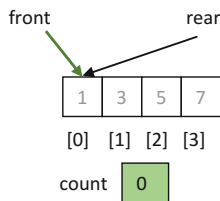
Up to this point, this section has only examined the enqueue operation, but what about the dequeue operation? One solution is to emulate the movie ticket example and move all of the elements in the array back from position 1 through 3 to position 0 through 2 and have `rear` adjusted accordingly. However, this would require the use of a loop and if there were n items in the array, it would be an $O(n)$ operation which was what was trying to be avoided at the beginning of this section.

Instead of having all the people in the line move forward one position, a better solution from the movie ticket example is to have the person issuing the tickets move down the line. In an array, instead of moving the items in the array and adjusting `rear`, it is far more efficient just to adjust `front`. In looking at the previous diagram, one should notice that when 1 is dequeued, `front` could be incremented by one to indicate the new front of the queue and `count` decremented by one to indicate one less item in the queue as shown below.

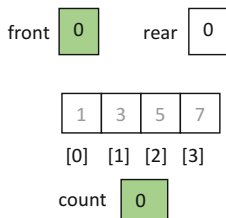


The result is that the dequeuer operation could be $O(1)$ instead of $O(n)$. As with stacks in the last chapter, the integer 1 is not actually removed from the array, but since `front` is no longer at position 0, the 1 is no longer part of the queue. Although in the future the dequeued item might not be shown, it is shaded light gray in this section to help make the distinction between the queue data structure and the array in which it is implemented.

This process of dequeuing would continue until all of the items have been dequeued, `front` and `rear` point to the same location (which does not need to be location 0), and `count` is again 0 indicating an empty queue.

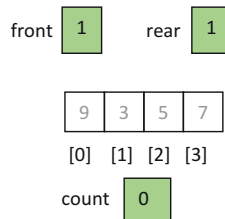


Alternatively, the following drawing indicates the same state.



Again, note that the items in the array are shaded light gray, indicating they are still in the array, but not in the queue. This is why checking the contents of the array to see if it is empty as suggested earlier is not a good indicator that the queue is empty.

Some programmers erroneously think when `front` and `rear` each contain a 0 it indicates an empty queue or possibly a full queue. However, an empty or full queue can occur anywhere in an array. For example, in the above diagram, consider enqueueing the integer 9 which would cause `rear` to become one and `count` would also become one. When it is dequeued, as shown in light gray below, `count` would be decremented to zero and `front` would become one. Both `front` and `rear` would now both equal 1, not 0, and yet the queue is empty. This is yet another reason for the inclusion of the variable `count` as shown below:

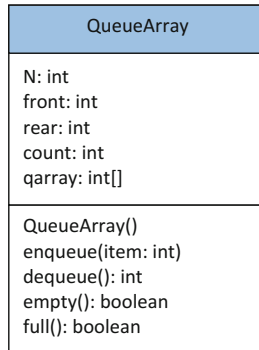


3.3 Queue Class: Data Members and Methods

Having looked at how a queue functions, the class for the queue can be created. To distinguish it from a queue implemented later in this text, the name of the class could be `QueueArray`. The data members for the queue can be seen in the drawings in the previous section. The queue needs to be implemented as an array which could be called `qarray`, and the three other data members `front`, `rear`, and `count`. Of course keeping with the practices of object-oriented programming, these data members should be declared as `private` and accessible only through `public` methods.

There also need to be four methods as discussed in the previous section plus a constructor. The four methods are `enqueue`, `dequeue`, `full`, and `empty`. The `enqueue` method will need a parameter to send the item to be enqueued and will be a `void` method since it does not return a value. The `dequeue` method will not need a parameter but will be a value-returning method to return the item dequeued. The `full` and `empty` methods will be value-returning methods returning the appropriate boolean value. The `enqueue` and `dequeue` methods should be `public` and the `empty` and `full` methods can be made `public` or `private` as needed.

As with a stack in Sect. 2.3, a UML diagram can be created to illustrate the `QueueArray` class as shown below:



The constructor will need to initialize the three integer variables to 0. The `enqueue` method will place the item into `qarray` at the location indicated by the index `rear`, then `rear` will need to be incremented by 1, and `count` will also need to be incremented by 1 as follows:

```
qarray[rear] = item;
rear = rear + 1;
count = count + 1;
```

Further, if `rear` is equal to the size of the array indicated by the constant `N`, then it will need to wrap around as discussed in the previous section. This can be accomplished by using the mod function after `rear` has been incremented as shown below:

```
qarray[rear] = item;
rear = (rear + 1) % N;
count++;
```

Note that `count` has been changed to the auto increment mode since it does not need to use the mod function. Of course if the array is full then the value in `item` should not be placed in the array and a message should be output.

```
if(full())
    System.out.println("Queue is full: item not enqueued");
else {
    qarray[rear] = item;
    rear = (rear + 1) % N;
    count++;
}
```

Alternatively, an exception could be used to indicate a full queue.

```
if(full())
    throw new RuntimeException("Queue is full: " +
        "item not enqueued");
qarray[rear] = item;
rear = (rear + 1) % N;
count++;
```

As in the previous chapter on stacks, note that the else section is not needed since the program will stop if an exception is thrown. The resulting complete enqueue method would appear as follows:

```
public void enqueue(int item) {
    if(full())
        throw new RuntimeException("Queue is full: " +
            "item not enqueued");
    qarray[rear] = item;
    rear = (rear + 1) % N;
    count++;
}
```

The dequeue operation is similar to the enqueue operation except the value of `item` must be taken from the front of the queue. Since `front` can wrap around the same as `rear` does in `enqueue`, note that the mod function is already included in the code segment. Lastly, `count` will need to be decremented by 1 as shown below:

```
item = qarray[front];
front = (front + 1) % N;
count--;
```

Whereas `enqueue` is concerned with whether the queue is full, `dequeue` must check whether the queue is empty and output the appropriate message.

```
if(empty())
    System.out.println("Queue is empty: item not dequeued");
else {
    item = qarray[front];
    front = (front + 1) % N;
    count--;
}
```

As before, an exception can be used in lieu of a `println` statement.

```
if(empty())
    throw new RuntimeException("Queue is empty: " +
        "item not dequeued");
item = qarray[front];
front = (front + 1) % N;
count--;
```

Once the item is removed from the queue, it will need to be returned to the calling program or method. As with stacks in the previous chapter, when using `if` and `println` statements the method will continue on to the `return` statement and `item` needs to contain a value. As before, the value `-1` is chosen and the complete `dequeue` method is as follows:

```
public int dequeue() {
    int item = -1;
    if(empty())
        System.out.println("Queue is empty: item not dequeued");
    else {
        item = qarray[front];
        front = (front + 1) % N;
        count--;
    }
    return item;
}
```

When using an exception, the program will terminate and the `return` statement will not be executed, so there is no need to include a value for `item`. The complete `dequeue` method using an exception is shown below.

```
public int dequeue() {
    int item;
    if(empty())
        throw new RuntimeException("Queue is empty: " +
            "item not dequeued");
    item = qarray[front];
    front = (front + 1) % N;
    count--;
    return item;
}
```

Lastly, the `full` and `empty` value-returning methods will be needed. As with stacks in the preceding chapter, `full` and `empty` can be written without the use of an `if` statement. Further, recall from the preceding section that when `count` equals `N`, then the queue is full and when `count` equals `0`, the queue is empty.

```
return count == N;           return count == 0;
```

Also, as with stacks previously, the relation can alternatively include checking for values greater than or equal to `N` or less than or equal to `0` as follows:

```
return count >= N;          return count <= 0;
```

The resulting two complete boolean value returning methods are:

```
public boolean full() {      public boolean empty() {
    return count >= N;        return count <= 0;
}                             }
```

Putting the data members and methods together, the complete `QueueArray` class is shown in Fig. 3.1.

Note that the `println` statements are used instead of the exceptions which will help facilitate the development of the main program in the next section. The methods using exceptions can be used instead merely by substituting the methods developed above.

3.4 Program Using the `QueueArray` Class

To test the `QueueArray` class, a simple program should be developed. This program will merely prompt the users whether they want to enqueue or dequeue an item. The letter `E` is used to indicate the former which is then followed by a prompt and input for the item to be enqueued. The letter `D` is used to indicate the latter and the item dequeued is displayed. To stop the program, the letter `S` is entered which serves as a sentinel value for a sentinel controlled loop.

In order to enter a single character, note that the following code segment would work assuming the appropriate declaration of the `Scanner` class. The character is input into the string variable `tempString` and then the single character is extracted from the string using `charAt(0)` and placed into the `char` variable `selection`.

```
String tempString;
char selection;
System.out.print("Please enter E for Enqueue, " +
```

```

public class QueueArray {
    private final int N = 4;
    private int front, rear, count, qarray[];
    public QueueArray() {
        front = rear = count = 0;
        qarray = new int[N];
    }
    public int dequeue() {
        int item = -1;
        if(empty())
            System.out.println("Queue is empty: " +
                               "item not dequeued");
        else {
            item = qarray[front];
            front = (front + 1) % N;
            count--;
        }
        return item;
    }
    public void enqueue(int item) {
        if(full())
            System.out.println("Queue is full: item not enqueued");
        else {
            qarray[rear] = item;
            rear = (rear + 1) % N;
            count++;
        }
    }
    public boolean empty() {
        return count <= 0;
    }
    public boolean full() {
        return count >= N;
    }
}

```

Fig. 3.1 QueueArray Class

```

"D for Dequeue, or S for stop: ");
tempString = scanner.next();
selection = tempString.charAt(0);

```

However, this could be shortened some by eliminating the `tempString` variable and invoking the `charAt(0)` method just after the `next` method as shown below:

```

char selection;
System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for stop: ");
selection = scanner.next().charAt(0);

```

This of course could be used in a sentinel controlled loop as follows:

```
System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for stop: ");
selection = scanner.next().charAt(0);
while (selection != 'S' && selection != 's') {
    // body of loop
    System.out.print("Please enter E for Enqueue, " +
"D for Dequeue, or S for stop: ");
    selection = scanner.next().charAt(0);
}
```

Note that there are both the priming read and the second read at the bottom of the loop as is typical for sentinel controlled loops. Also notice that the `while` statement checks for both uppercase `S` and lowercase `s` should a user forget to press the shift key on the keyboard.

Once the loop is created, the logic for the other two possible inputs can be created. As mentioned previously, should the letter `E` be entered, then the user should be prompted for the item to be enqueued and then the `enqueue` method should be invoked. If the queue is full, then the `enqueue` method will output the appropriate message. Should the user enter the letter `D` then the item at the front of the queue should be dequeued and output. If the queue is empty, then the `dequeue` method will also output the appropriate message and return a `-1` which should not be output by the main program. If any other value is entered, a user-friendly message should be output by the main program.

```
if(selection == 'E' || selection == 'e') {
    System.out.print("Please enter an integer to be enqueued: ");
    item = scanner.nextInt();
    queue.enqueue(item);
}
else
    if(selection == 'D' || selection == 'd') {
        item=queue.dequeue();
        if(item != -1)
            System.out.println("The item dequeued is: " + item);
    }
else
    System.out.println("The selection entered " + selection
    + " is not E, D, or S, please try again");
```

Putting the above code inside the body of the loop and including all the necessary declarations, the complete program looks as follows:


```
import java.util.*;
public class TestQueueArray {
    public static void main(String args[]) {
        int item;
        char selection;
        QueueArray queue;
        queue = new QueueArray();
        Scanner scanner = new Scanner(System.in);

        System.out.println();
        System.out.print("Please enter E for Enqueue, " +
            "D for Dequeue, or S for Stop: ");
        selection = scanner.next().charAt(0);
        while(selection != 'S' && selection != 's') {
            if(selection == 'E' || selection == 'e') {
                System.out.print("Please enter an integer to be " +
                    "enqueued: ");
                item = scanner.nextInt();
                queue.enqueue(item);
            }
            else
                if(selection == 'D' || selection == 'd') {
                    item = queue.dequeue();
                    if(item != -1)
                        System.out.println("The item dequeued is: " +
                            item);
                }
            else
                System.out.println("The selection entered " +
                    selection + " is not E, D, or S, please try again");
            System.out.println();
            System.out.print("Please enter E for Enqueue, " +
                "D for Dequeue, or S for Stop: ");
            selection = scanner.next().charAt(0);
        }
        System.out.println();
        System.out.println("End of Program");
        System.out.println();
    }
}
```

Sample output from the program showing all the messages including prompting for a selection, prompting for an item to enqueue, the dequeued output, and the messages that the queue is full, the queue is empty, wrong selection, and the end of program message is as follows:

Please enter E for Enqueue, D for Dequeue, or S for Stop: q
The selection entered q is not E, D, or S, please try again

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 1

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 2

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 3

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 4

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 5
Queue is full: item not enqueued

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 1

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 5

Please enter E for Enqueue, D for Dequeue, or S for Stop: e
Please enter an integer to be enqueued: 6
Queue is full: item not enqueued

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 2

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 3

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 4

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
The item dequeued is: 5

Please enter E for Enqueue, D for Dequeue, or S for Stop: d
Queue is empty: item not dequeued

Please enter E for Enqueue, D for Dequeue, or S for Stop: s

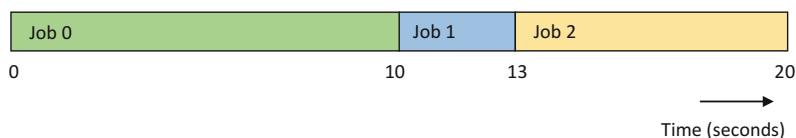
End of Program

3.5 Complete Program: Simulating a Scheduling Algorithm

In this section a simple scheduler in an operating system is simulated using an array-based queue. As mentioned in Sect. 3.1, queues are used in many instances in computer software. Often on a computer more than one application, such as a word processor, a spreadsheet, and a web browser, can be opened at the same time. Assuming the machine has one processor, these programs, sometimes called jobs, are submitted to the operating system for execution. A *CPU scheduler* can then decide which program will run next. There are various scheduling algorithms, but one of the simplest uses a First-In, First-Out queue described in this chapter. With a queue, the job that requests the CPU first is allocated the CPU first. Consider the following set of jobs with the length of the CPU time in seconds that is required for finishing the job.

Job Number	CPU Time Required
0	10
1	3
2	7

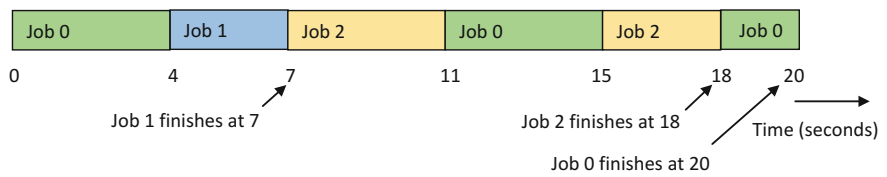
If the jobs arrive in the order Job 0 through Job 2 when the CPU is idle and are served in FIFO fashion, the timeline of how these jobs are executed can be shown in the following chart called a Gantt chart:



To make it simple, all the jobs are submitted at the beginning and no jobs will arrive later. As soon as Job 0 enters the queue, it will be dequeued for execution and finishes after 10 s. After waiting for 10 s for Job 0 to finish, Job 1 is dequeued, the CPU is allocated the job, and it completes after 3 s at the 13 s mark. Job 2 waits for 13 s in the queue while Job 0 and Job 1 are executing. It is then dequeued, enters the CPU, and finishes after 7 s at the 20 s mark. Although all the jobs are processed, the disadvantage of this method is that shorter jobs arriving after the longer jobs have to wait a longer time although they do not require a longer CPU time.

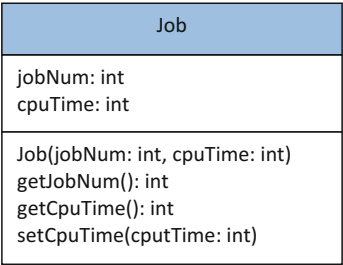
For every job to have a fair waiting time, a *round-robin* (RR) scheduling algorithm can be used. It is basically FIFO scheduling, but each job can only stay in the CPU a certain amount of time. If it does not finish during the time given, it simply gives up the CPU and reenters the queue in order to receive further CPU time later. A small unit of time, called a time quantum, is used. Should a job entering the CPU have a CPU time of less than the time quantum, the job will release the CPU and the scheduler will then proceed to the next job in the queue.

Otherwise, if the CPU time of the currently running job is longer than the time quantum, the job will be put at the rear of the queue. The CPU scheduler will then select the next job in the queue. Using the same example above and a quantum of 4 s the Gantt chart will look like the following:



Job 0 gets the first 4 s which is the size of the quantum. Since it requires another 6 s to complete, it is preempted and placed at the rear of the queue, and then the CPU is given to the next job in the queue, Job 1. Job 1 does not need 4 s, so it finishes before its time quantum expires. The CPU is then given to the next job, Job 2. Once each job has received one time quantum, the CPU is returned to Job 0 at the 11 s mark for an additional time quantum. As can be seen, Job 1 finishes at the 7 s mark, Job 2 at the 18 s mark, and Job 0 at the 20 s mark. The advantage is that the shorter job does not have to wait as long.

Given the preceding brief discussion of CPU scheduling algorithms, a round-robin scheduler will be implemented using an array-based queue to find the time when each job completes. With this application the items stored in the queue are representations of the jobs, rather than simple integers discussed in the previous sections, which can be implemented as objects with data and operations. So, the Job class will need to be defined first. Two values will be kept in a Job object, jobNum and cpuTime. The jobNum is the ID number of each job and the cpuTime is the time CPU requires to complete the job. The class will contain one constructor that accepts two parameters to initialize both data members. Two accessors are defined to get the values of the two data members and a mutator to set the value of the cpuTime. The following UML diagram represents the Job class.



As can be seen, there is no mutator for the jobNum since the value does not change once it is assigned; however, cpuTime will need to be updated during the execution using the setCpuTime method. The code defining the class for a Job object is shown below:

```
public class Job {
    private int jobNum, cpuTime;

    // constructor
    public Job(int jobNum, int cpuTime) {
        this.jobNum = jobNum;
        this.cpuTime = cpuTime;
    }

    // returns the jobNum
    public int getJobNum() {
        return jobNum;
    }

    // returns the cpuTime
    public int getCpuTime() {
        return cpuTime;
    }

    // sets the cpuTime
    public void setCpuTime(int cpuTime) {
        this.cpuTime = cpuTime;
    }
}
```

Since objects of `Job` class are kept in the queue, the `QueueArray` class shown in Fig. 3.1 needs to be rewritten for the `Job` class. Although it is not difficult to modify the `QueueArray` class for the `Job` object, rewriting a `QueueArray` class for the specific object every time a new application is developed is not very efficient. Remember in Sect. 2.5 where generic types were discussed? The `QueueArray` class that can be used for any objects will be written next. Essentially the `int` in `QueueArray` is replaced by the type parameter `T`. Figure 3.2 shows the complete `QueueArrayGeneric` class definition. Notice that the public method, `getCount` is added in order to get the value of the private data member `count` of the `QueueArrayGeneric` class in the main program. In the `dequeue` method, the local variable `item` is initialized to `null`, indicating that initially no reference to a `Job` object is assigned to the variable. Therefore, instead of returning a `-1`, a value of `null` is returned when the queue is empty.

In the main program, two methods, `setJobs` and `runScheduler` are invoked. The `setJobs` method is to obtain information from a user about the jobs to be executed in the CPU. The `runScheduler` simulates the round-robin scheduler. The main program without the detailed implementation of the two methods is shown below, and the two methods are defined shortly.

```

public class QueueArrayGeneric<T> {
    private final int N = 5;
    private int front, rear, count;
    private T[] qarray;

    public QueueArrayGeneric() {
        front = rear = count = 0;
        qarray = (T[]) new Object[N];
    }

    public T dequeue() {
        T item = null;
        if(empty())
            System.out.println("Queue is empty: item not dequeued");
        else {
            item = qarray[front];
            front = (front + 1) % N;
            count--;
        }
        return item;
    }

    public void enqueue(T item) {
        if(full())
            System.out.println("Queue is full: item not enqueued");
        else {
            qarray[rear] = item;
            rear = (rear + 1) % N;
            count++;
        }
    }

    public boolean empty() {
        return count <= 0;
    }

    public boolean full() {
        return count >= N;
    }

    public int getCount() {
        return count;
    }
}

```

Fig. 3.2 QueueArrayGeneric < T > class

```

import java.util.*;

public class RoundRobin {
    private static Scanner scanner = new Scanner(System.in);
    private static QueueArrayGeneric<Job> jobQueue
        = new QueueArrayGeneric<Job>();
    private static int quantum;

```

```
public static void main(String[] args) {
    setJobs();
    runScheduler();
}

// gets information about jobs run in CPU
public static void setJobs() {
    // defined shortly
}

// simulates the round-robin scheduler
public static void runScheduler() {
    // defined shortly
}
}
```

In the `setJobs` method, first a user will enter both the number of jobs and size of the time quantum. Then for the number of jobs entered, the user will be prompted for the amount of CPU time needed for each job, a `Job` object will be created, and then placed in the queue. The complete definition of the `setJobs` method is given below:

```
// obtains information from a user about jobs to be run in CPU
public static void setJobs() {
    Job job;
    int jobNum, cpuTime;

    System.out.print("Enter number of jobs: ");
    jobNum = scanner.nextInt();
    System.out.print("Enter size of quantum: ");
    quantum = scanner.nextInt();
    System.out.println();

    for(int i=0; i<jobNum; i++) {
        System.out.print("Enter CPU time for job " + i + ": ");
        cpuTime = scanner.nextInt();
        job = new Job(i, cpuTime);
        jobQueue.enqueue(job);
    }
    System.out.println();
}
```

In the second method, `runScheduler`, the program will continue to execute until the queue becomes empty indicating that all of the jobs are finished. Inside the while loop, after a job in the queue is dequeued to be in the CPU, the program checks if the job will finish before the quantum expires. If it will not, the value of `cpuTime` of the job is updated by subtracting the time quantum from the current value, and then the job reenters the queue. Otherwise, it outputs the message saying the job is completed. Notice that the variable `accumulatedCpuTime` keeps track of the time the CPU spent executing jobs so far. It is updated at the beginning of both the then and the else sections with the value of either quantum or `cpuTime`, respectively. The complete implementation of `runScheduler` is shown below:

```
// simulates the round-robin scheduler
public static void runScheduler() {
    Job job;
    int cpuTime, accumulatedCpuTime;
    accumulatedCpuTime = 0;

    while(!jobQueue.empty()) {
        // front of the queue goes to the CPU
        job = jobQueue.dequeue();
        cpuTime = job.getCpuTime();

        // quantum expires before job finishes
        if(cpuTime - quantum > 0) {
            accumulatedCpuTime = accumulatedCpuTime + quantum;
            // update cpuTime of job and put it back in the queue
            job.setCpuTime(cpuTime - quantum);
            jobQueue.enqueue(job);
        }
        // job finishes before time quantum
        else {
            accumulatedCpuTime = accumulatedCpuTime + cpuTime;
            // job is done, output a message
            System.out.println("Job " + job.getJobNum()
                + " finished at " + accumulatedCpuTime);
        }
    }
}
```

When the above program is compiled and executed using the sample input, the output of the program looks as given below:


```
Enter number of jobs: 3
Enter size of quantum: 4

Enter CPU time for job 0: 10
Enter CPU time for job 1: 3
Enter CPU time for job 2: 7

Job 1 finished at 7
Job 2 finished at 18
Job 0 finished at 20
```

3.6 Summary

- A queue is a First-In, First-Out (FIFO) data structure.
- Putting an item in a queue is known as an enqueue operation and removing an item from a queue is known as a dequeue operation.
- An enqueue operation places an item at the rear of a queue and a dequeue operation removes an item from the front of a queue.
- It is much faster to move a front or rear index which is $O(1)$ than to have to move the items in a queue which is $O(n)$.
- The mod function % is helpful in implementing the wraparound feature of a queue.
- A count of the number of items in a queue is an easy way of checking for a full or empty queue.
- Generic types make queues reusable with different types of objects.

3.7 Exercises (Items Marked with an * Have Solutions in Appendix B)

- *1. Assuming the queue `intQueue` of type `QueuekArray` in Fig. 3.1 is initially empty, draw a queue after the following operations are completed:

```
int x, y, z;
intQueue.enqueue(41);
intQueue.enqueue(6);
x = intQueue.dequeue();
y = intQueue.dequeue();
intQueue.enqueue(78);
intQueue.enqueue(5);
intQueue.enqueue(79);
```

```
z = intQueue.dequeue();
intQueue.enqueue(5);
```

2. Assuming the queue `characterQueue` of type `QueueArrayGeneric < Character>` which is an object of the generic class `QueueArrayGeneric <T>` in Fig. 3.2 is initially empty, draw a queue after the following operations are completed:

```
Character x, y, z;
characterQueue.enqueue('U');
characterQueue.enqueue('V');
x = characterQueue.dequeue();
characterQueue.enqueue('W');
y = characterQueue.dequeue();
characterQueue.enqueue('P');
characterQueue.enqueue('Q');
z = characterQueue.dequeue();
characterQueue.enqueue('C');
```

3. Assuming the queue `intQueue` of type `QueueArray` in Fig. 3.1 is declared and several items are already enqueued, and also suppose the value of `front` is 3 and the value of `rear` is 0, answer the following questions.
 - *A. What are the values of `count`, `front` and `rear` after adding an element to `intQueue`?
 - B. What are the values of `count`, `front` and `rear` after removing an element from `intQueue`?
- *4. Write a method `peek` that can be added to the `QueueArray` class in Fig. 3.1 which returns the item at the beginning of the queue without removing it.
5. In Sect. 2.7 the concept of a palindrome was introduced. A palindrome is a sequence of symbols such as a word, phrase, verse, or sentence, that reads the same way from either direction, forward or backward. Write a program that reads a string, places each character of the string in both a queue and a stack, and then verifies whether the string is a palindrome.
6. Write a method that takes a stack as a parameter and reverse the elements of the stack using a queue whose elements are of the same type.
7. Write a method that takes a queue as a parameter and reverse the elements of the queue using a stack whose elements are of the same type.
8. Some automobiles have the capability to determine the average miles per gallon (MPG) over the last fifty miles driven. In order to simulate this, assume that the MPG can be input once every mile and an average MPG is calculated over the last five miles instead of the last fifty. If fewer than five miles have been input, the average should be over the number of MPGs input. A five-element

array-based queue should be used for this problem, and further, all of the MPGs should not be re-summed each time a new MPG is input. Below is some sample input (current MPG) and output (average MPG) with a negative number that terminates the program.

```
Enter Current MPG: 25
The Average MPG is: 25.0
Enter Current MPG: 30
The Average MPG is: 27.5
Enter Current MPG: 40
The Average MPG is: 31.7
Enter Current MPG: 30
The Average MPG is: 31.3
Enter Current MPG: 40
The Average MPG is: 33.0
Enter Current MPG: 30
The Average MPG is: 34.0
Enter Current MPG: 40
The Average MPG is: 36.0
Enter Current MPG: 40
The Average MPG is: 36.0
Enter Current MPG: 35
The Average MPG is: 37.0
Enter Current MPG: -1
```

9. CPU scheduling algorithms were discussed in Sect. 3.5. Using the complete program as a basis, modify it to find an average waiting time of all the jobs entered in the queue at the 0 second mark, where a waiting time is the total time the job spends in the queue before it finishes executing. Find average waiting times using a FIFO queue and a round-robin scheduler with the same set of jobs.